Precisão de Estado e Decaimento sem Multiplicador no *Scan* Seletivo do Mamba-2 em Hardware

André Goveia[1]

Programa de Pós-Graduação – Aplicações de Alto Desempenho
andre@instituicao.edu.br

Abstract. *Language and signal models based on the Transformer must keep every past input, so their memory grows with the input length. State space models such as Mamba-2 keep a fixed-size summary of the past instead, updated at every step like a bank of first-order recursive (IIR) filters whose coefficients depend on the input. This update, the* selective scan*, is the part of the model that hardware must run step by step, and it multiplies every state element by a decay coefficient at every step. The coefficients that give the model long memory sit very close to 1, where cheap shift-only arithmetic fails, and short fixed-point state registers stop decaying, an effect analogous to the dead band of fixed-point IIR filters. This paper asks whether the decay multiplier can be removed without losing accuracy, and how much state precision is needed. We use the complement power-of-two decay $1 - 2^{-k}$, applied with one shift and one subtraction (the same form used by QMamba for post-training quantization and by shift-based moving averages in DSP), in a per-token bimodal form $\{2^{-k}, 1 - 2^{-k}\}$ that we call CPoT, and quantify the state precision it requires in an integer scan. A 16-bit truncating state reaches only 37,0 dB of datapath SQNR, while a 32-bit state with 10 guard bits and rounding reaches 99,0 dB, on par with a Q1.15 multiplier (99,3 dB). Models trained with the hardware constraints and evaluated with the bit-exact integer scan keep FP32 accuracy on ECG5000 and short-range recall, and match the multiplier baseline on long-range recall; on a task that requires gradual forgetting, CPoT reaches 89,9% against 83,0% for plain power-of-two decay and 90,5% for the multiplier baseline, while the naive 16-bit state drops to 83,4% (worse in all 10 seeds) and solves none of the long-range recall seeds. On an Intel Cyclone V, used as a measurement vehicle, the CPoT scan lane uses 48 DSP blocks instead of 80 for the same throughput, at the cost of more logic and, in a vectorless estimate, more dynamic power. In a 45 nm open-source ASIC flow (Nangate45, OpenROAD) at the same 250 MHz clock, it is 43% smaller and uses 28% less energy per step than the multiplier lane, with a lower maximum frequency (306 vs. 366 MHz). The RTL is checked bit-exactly against an integer reference model (6.144 outputs, 0 mismatches).*

Resumo. *Modelos de linguagem e de sinais baseados no Transformer precisam guardar todas as entradas passadas, e sua memória cresce com o comprimento da entrada. Modelos de espaço de estados como o Mamba-2 guardam um resumo de tamanho fixo do passado, atualizado a cada passo como um banco de filtros recursivos (IIR) de primeira ordem cujos coeficientes dependem da entrada. Essa atualização, o* scan seletivo*, é a parte do modelo que o hardware precisa executar passo a passo, e ela multiplica cada elemento do estado por*

um coeficiente de decaimento a cada passo. Os coeficientes que dão memória longa ao modelo ficam muito próximos de 1, onde a aritmética só com deslocamentos falha, e registradores de estado curtos em ponto fixo param de decair, efeito análogo à zona morta de filtros IIR em ponto fixo. Este artigo investiga se o multiplicador do decaimento pode ser eliminado sem perda de acurácia e quanta precisão o estado precisa. Usamos o decaimento pelo complemento em potência de dois, $1 - 2^{-k}$*, aplicado com um deslocamento e uma subtração (a mesma forma usada pelo QMamba na quantização pós-treino e pelas médias móveis por deslocamento em DSP), numa forma bimodal por passo,* $\{2^{-k}, 1 - 2^{-k}\}$*, que chamamos de CPoT, e quantificamos a precisão de estado que ele exige num* scan *inteiro. Um estado de 16 bits com truncamento atinge só 37,0 dB de SQNR no* datapath*, enquanto um estado de 32 bits com 10 bits de guarda e arredondamento atinge 99,0 dB, no nível de um multiplicador Q1.15 (99,3 dB). Modelos treinados com as restrições do hardware e avaliados com o* scan *inteiro bit-exato mantêm a acurácia FP32 no ECG5000 e na recordação de curto alcance, e empatam com o baseline com multiplicador na de longo alcance; numa tarefa que exige esquecimento gradual, o CPoT atinge 89,9%, contra 83,0% da potência de dois simples e 90,5% do baseline com multiplicador, enquanto o estado ingênuo de 16 bits cai para 83,4% (pior nas 10 sementes) e não resolve nenhuma semente da recordação de longo alcance. Numa Intel Cyclone V, usada como veículo de medida, a* lane *de* scan *CPoT usa 48 blocos DSP em vez de 80 para a mesma vazão, ao custo de mais lógica e, numa estimativa sem simulação, de mais potência dinâmica. Num fluxo ASIC aberto de 45 nm (Nangate45, OpenROAD), na mesma frequência de 250 MHz, ela ocupa 43% menos área e gasta 28% menos energia por passo que a* lane *com multiplicador, com frequência máxima menor (306 contra 366 MHz). O RTL foi conferido bit a bit contra um modelo de referência inteiro (6.144 saídas, 0 divergências).*

1 Introdução

1.1 Motivação: memória de tamanho fixo

Modelos de sequência processam entradas que chegam uma após a outra: palavras de um texto, amostras de um eletrocardiograma, quadros de áudio. O Transformer, base dos modelos de linguagem atuais, processa cada nova entrada consultando *todas* as anteriores (mecanismo de atenção) e, para isso, guarda uma cópia de cada uma delas, o *KV-cache*. Essa memória cresce linearmente com o comprimento L da sequência: mesmo um Transformer pequeno, com a largura do modelo usado neste trabalho (2 camadas, 64 canais, valores em INT8), guardaria $2 \cdot 2 \cdot 64 = 256$ bytes por passo, ou 1 MiB para $L = 4.096$.

Modelos de espaço de estados seletivos, como o Mamba [9] e o Mamba-2 [6], não guardam o passado; guardam um *estado* de tamanho fixo que o resume e é atualizado a cada passo. Para quem vem de hardware, a forma mais simples de entender esse estado é como um filtro recursivo (IIR) de primeira ordem, $h_t = a\, h_{t-1} + b\, x_t$: o coeficiente a define por quanto tempo uma entrada continua influenciando o estado (com $a = 0{,}999$, cerca de mil passos). O Mamba-2 é um banco de milhares desses filtros em paralelo, com uma diferença importante: a e b mudam a cada passo em função da própria entrada, de

modo que o modelo decide o que lembrar e o que esquecer. No modelo usado aqui o estado ocupa 16 KiB qualquer que seja L.

1.2 O problema, em termos de hardware

A atualização do estado, chamada de *scan seletivo*, é a parte do Mamba que precisa ser executada passo a passo: h_t depende de h_{t-1}. Em GPUs, o treino e o processamento de um texto de entrada inteiro reescrevem o *scan* como multiplicações de matrizes em blocos [6]; mas na geração token a token, em aceleradores dedicados e em FPGAs, ele é executado como recorrência, por uma unidade de hardware própria [14, 22, 21]. Essa unidade enfrenta três obstáculos:

1. **Um multiplicador por elemento do estado.** A cada passo, cada elemento do estado é multiplicado pelo coeficiente de decaimento. Com milhares de elementos, são milhares de multiplicações por passo, e multiplicadores são o recurso mais caro (blocos DSP em FPGAs, área e energia em ASICs).

2. **Coeficientes próximos de 1.** Os canais de memória longa têm a entre $0{,}90$ e $0{,}999$. A troca óbvia, aproximar a por uma potência de dois ($a \approx 2^{-k}$) e multiplicar com um deslocamento de bits, falha: não existe potência de dois entre $0{,}5$ e 1, e todos esses canais perdem a memória.

3. **Precisão do estado.** Mesmo com o coeficiente certo, um estado em ponto fixo curto com deslocamento truncado para de decair quando fica pequeno. É o mesmo fenômeno da *zona morta* de filtros IIR em ponto fixo, conhecido em processamento digital de sinais: o canal "congela" num valor que deveria ter sido esquecido.

1.3 Objetivo e questões de pesquisa

O objetivo deste trabalho é **eliminar o multiplicador do decaimento na unidade de *scan* do Mamba-2, substituindo-o por deslocamento e subtração, sem perda de acurácia em relação ao modelo em ponto flutuante, e medir quanto isso economiza em hardware**. Isso se desdobra em quatro questões:

QP1 (coeficiente). Que aproximação do coeficiente por deslocamentos preserva memórias longas?

QP2 (precisão). Quantos bits, e que tipo de arredondamento, o estado inteiro precisa para que o decaimento funcione?

QP3 (acurácia). Modelos treinados com o *scan* inteiro proposto mantêm a acurácia FP32? A aproximação proposta supera a potência de dois simples em tarefas que exigem memória de duração intermediária?

QP4 (custo). Quanto a unidade proposta custa em blocos DSP, lógica, frequência, área e energia, comparada a uma unidade com multiplicador, numa FPGA e num processo ASIC?

1.4 Contribuições e escopo

O que não é novo. Aproximar um coeficiente próximo de 1 por $1 - 2^{-k}$ e aplicá-lo como $h - (h \gg k)$ é a média móvel exponencial por deslocamento, clássica em DSP, e já foi usada no Mamba pelo QMamba [15], na quantização pós-treino de modelos de visão. A zona morta de filtros IIR em ponto fixo também é conhecida [19]. O que este trabalho acrescenta é o que falta para essa forma funcionar num *scan* inteiro em hardware.

Contribuições. (1) Uma análise da precisão do estado inteiro no *scan* seletivo com decaimento por deslocamento: a implementação ingênua (16 bits, truncamento) congela os canais de memória longa, o efeito aparece na acurácia de modelos treinados, e 32 bits com 10 bits de guarda e arredondamento o eliminam. Não encontramos essa análise em trabalhos de SSM, incluindo os que usam o deslocamento. (2) Uma *lane* de *scan* parametrizável em Verilog, sem multiplicador no decaimento, conferida bit a bit contra um modelo de referência e medida contra uma *lane* com multiplicador numa FPGA Intel Cyclone V (Quartus) e num processo ASIC de 45 nm (Yosys e OpenROAD). (3) Um *scan* treinado já com as restrições do hardware (escalas em potência de dois, passo $\Delta_t = 2^{-j}$, decaimento CPoT), avaliado com a aritmética bit-exata do RTL em dados reais e em tarefas de memória, incluindo uma tarefa que exige esquecimento gradual e separa o complemento da potência de dois simples. (4) Dois refinamentos de projeto: o código bimodal por passo $\{2^{-k}, 1 - 2^{-k}\}$ (CPoT) e a obtenção do código (m, k) por uma tabela de 16 entradas indexada pelo expoente de Δ_t, sem calcular $\exp$ em hardware.

Escopo. O objeto de estudo é a unidade de *scan*. As projeções lineares do modelo usam INT8 convencional (W8A8), como em aceleradores Mamba existentes [21], e não são analisadas. A FPGA é usada como veículo de medida: síntese, posicionamento e análise de *timing* no Intel Quartus dão números reais de blocos DSP, lógica e frequência para comparar variantes da unidade no mesmo dispositivo. Para área e energia, que são as métricas naturais de uma unidade dentro de uma GPU ou NPU, o mesmo RTL passa por um fluxo ASIC aberto (Yosys e OpenROAD) com a biblioteca Nangate45. Os números das duas plataformas valem como comparação relativa entre variantes; não há execução em placa nem chip fabricado.

Organização. A Seção 2 apresenta os conceitos necessários; a Seção 3, os trabalhos relacionados; a Seção 4, a unidade de *scan*; a Seção 5, a metodologia em detalhe; a Seção 6, os resultados, organizados pelas questões de pesquisa; e as Seções 7 e 8, limitações e conclusões.

2 Fundamentação

2.1 Do filtro de primeira ordem ao Mamba-2

Um filtro, um estado. Considere o filtro $h_t = a\, h_{t-1} + b\, x_t$ com $0 < a < 1$. Uma entrada isolada em $t = 0$ deixa no estado o valor $b\, x_0\, a^t$ depois de t passos: decai exponencialmente. A *constante de tempo* $\tau = -1/\ln a$ é o número de passos até essa influência cair a $1/e \approx 37\%$. Para a próximo de 1, $\tau \approx 1/(1 - a)$.

Exemplo (constantes de tempo). $a = 0{,}5$ *dá* $\tau \approx 1{,}4$ *passos;* $a = 0{,}9$ *dá* $\tau \approx 9{,}5$*;* $a = 0{,}999$ *dá* $\tau \approx 1.000$*. Um canal que precisa lembrar algo visto mil passos atrás*

precisa de a na terceira casa decimal de 1.

O Mamba-2. O Mamba-2 [6] organiza muitos desses filtros em H *cabeças*. Cada cabeça h tem P canais de entrada, e cada canal tem N estados. Para o canal p no passo t:

$$\bar{A}_t = \exp(-\Delta_t A_h), \qquad \mathbf{h}_t = \bar{A}_t\, \mathbf{h}_{t-1} + \Delta_t\, x_t\, \mathbf{B}_t, \qquad y_t = \mathbf{C}_t^\top \mathbf{h}_t + D_h x_t, \quad (1)$$

com $\mathbf{h}_t, \mathbf{B}_t, \mathbf{C}_t \in \mathbb{R}^N$. Lendo cada termo:

- x_t é a entrada do canal e y_t a sua saída;
- $\bar{A}_t$ é o coeficiente a do filtro, comum aos N estados da cabeça; $A_h > 0$ é aprendido e fixo após o treino;
- $\Delta_t > 0$ é o *passo de tempo*, calculado a partir da entrada. Δ_t grande faz $\bar{A}_t$ pequeno (esquecer depressa) e escreve a entrada com mais força; Δ_t pequeno faz o estado ignorar a entrada e lembrar o passado;
- $\mathbf{B}_t$ diz em quais dos N estados escrever, e $\mathbf{C}_t$ quais ler; ambos também vêm da entrada.

Como Δ_t, $\mathbf{B}_t$ e $\mathbf{C}_t$ dependem da entrada, o modelo é *seletivo*: decide, a cada passo, o que guardar. Executar a Eq. 1 ao longo da sequência é o *scan*. No modelo usado aqui ($H = 4$, $P = 32$, $N = 16$), cada camada tem $4 \cdot 32 \cdot 16 = 2.048$ elementos de estado, e cada um é multiplicado por $\bar{A}_t$ a cada passo.

O bloco completo. Um bloco Mamba-2 recebe um vetor de D canais por passo e: (1) aplica uma projeção linear de entrada que produz x, $\mathbf{B}$, $\mathbf{C}$, Δ e um sinal de *gate* z; (2) passa x por uma convolução causal curta e pela não linearidade SiLU; (3) executa o *scan*; (4) multiplica y por $\mathrm{SiLU}(z)$; e (5) aplica a projeção de saída e soma o resultado à entrada. Este trabalho trata apenas do passo (3).

2.2 Quantização

Quantizar é representar um número real v por um inteiro q e uma escala s, com $v \approx s\,q$. Em INT8, $q \in [-127, 127]$. Se s é uma potência de dois, mudar de escala é um deslocamento de bits. Para que o modelo funcione bem depois de quantizado, usamos *quantization-aware training* (QAT): a quantização é simulada durante o treino, e o modelo aprende parâmetros que funcionam bem depois de arredondados. Como o arredondamento tem derivada zero, usamos o estimador *straight-through* (STE): na ida o valor é arredondado; na volta, o gradiente passa como se o arredondamento fosse a identidade.

2.3 Coeficiente com deslocamentos: PoT e CPoT

Multiplicar por 2^{-k} é deslocar k bits à direita ($h \gg k$), operação praticamente gratuita em hardware. Aproximar $\bar{A}_t$ por uma potência de dois (PoT) tem um problema na faixa que interessa: os valores possíveis são $1, 0{,}5, 0{,}25, \ldots$, e não há nenhum entre $0{,}5$ e 1. Com $k \geq 1$, todo canal lento vira $0{,}5$ ($\tau \approx 1{,}4$ passos); com $k \geq 0$, vira 1 e nunca esquece.

A saída é aproximar por potência de dois não o coeficiente, mas o seu *complemento* $1 - \bar{A}_t$, que é pequeno exatamente nos canais lentos. A ideia não é nova:

$y \mathrel{+}= (x - y) \gg k$ é a forma usual de uma média móvel exponencial em microcontroladores e DSPs, e o QMamba [15] a usa para quantizar $\bar{A}$ após o treino em modelos de visão (*log-to-shift quantization*, escolhida por tensor quando a mediana de $\bar{A}$ passa de $0{,}9$). Aqui usamos uma variante bimodal escolhida *por passo*, que chamamos de CPoT (*complementary power-of-two*). Como $\exp(-\Delta_t A_h) \approx 1 - \Delta_t A_h$ quando o produto é pequeno:

$$\bar{A}^{\mathrm{CPoT}}(m,k) = \begin{cases} 2^{-k}, & m = 0 \;\; (\bar{A}_t \le 0{,}625) \\ 1 - 2^{-k}, & m = 1 \;\; (\bar{A}_t > 0{,}625) \end{cases} \qquad \bar{A}^{\mathrm{CPoT}} h = \begin{cases} h \gg k \\ h - (h \gg k) \end{cases}, \tag{2}$$

com $k \in \{1, \ldots, 15\}$ escolhido por arredondamento no domínio logarítmico de cada ramo. O ramo $m = 1$ custa um deslocamento e uma subtração.

Exemplo (CPoT). *$\bar{A}_t = 0{,}998$ tem complemento $0{,}002 \approx 2^{-9}$, então $k = 9$ e $\bar{A}^{\mathrm{CPoT}} = 1 - 2^{-9} \approx 0{,}99805$ ($\tau = 512$ contra 500 do valor exato). Na faixa $[0{,}90;\ 0{,}999]$ o CPoT tem 8 níveis, de $k = 3$ a $k = 10$, um por oitava de τ.*

Como os níveis ficam uma oitava de τ distantes, o erro de τ pode chegar a cerca de 40%. A variante CPoT-2 ($1 - 2^{-k_1} \mp 2^{-k_2}$) usa um segundo deslocamento para refinar e foi avaliada só em software. Como também quantizamos $\Delta_t = 2^{-j}$ e A_h é fixo depois do treino, o código (m, k) de cada cabeça depende só de j: é uma tabela de 16 entradas, e o hardware não precisa calcular $\exp$.

2.4 Precisão necessária no estado

Com estado inteiro e deslocamento truncado, $h \gg k = 0$ sempre que $0 \le h < 2^k$. Nesse caso $h - (h \gg k) = h$: o decaimento para e o canal congela. É a zona morta (*deadband*) de filtros IIR em ponto fixo [19], aqui no pior caso, porque o deslocamento descarta todos os k bits de baixo. Para $h < 0$ o deslocamento aritmético dá -1 e o estado sobe de 1 em 1 até zero, o que cria uma assimetria entre valores positivos e negativos.

Exemplo (zona morta). *Num estado de 16 bits sem bits de guarda, h é medido em unidades do produto $B_t x_t$, que vai até ± 16.129 com entradas INT8. Com $k = 10$, qualquer $0 \le h < 2^{10} = 1.024$ congela: cerca de 6% do maior valor que pode ser injetado. Um impulso de 4.096 unidades deveria decair para 75 em 4τ passos, mas para em 1.023.*

A correção combina três medidas: (i) *G bits de guarda*, isto é, o estado guarda $h \cdot 2^G$ e ganha G bits fracionários; (ii) arredondamento para o mais próximo, $s = (h + 2^{k-1}) \gg k$, que torna o comportamento simétrico; e (iii) estado de 32 bits com saturação, para que os bits de guarda não reduzam a faixa. A zona morta não desaparece, mas cai para $|h| < 2^{k-1}$ na escala com guarda, ou seja, 2^{k-1-G} unidades do produto $B_t x_t$: meia unidade para $k = G = 10$. A injeção $\Delta_t x_t B_t$ vira $((x_t B_t) \ll G) \gg j$, e a leitura, $y_t = (\sum_n C_t[n]\, h_t[n]) \gg G$.

2.5 Recursos de uma FPGA Cyclone V

A FPGA usada como veículo de medida, a Intel Cyclone V `5CSEBA6U23I7` (placa DE10-Nano), tem três tipos de recurso relevantes [11]. **ALMs** (41.910) são os blocos lógicos: cada um tem uma tabela de consulta adaptativa que implementa uma função

de até 6 entradas (ou duas menores), somadores dedicados e registradores. **Blocos DSP** (112) contêm multiplicadores de precisão variável: três de 9×9 bits, dois de 18×18 ou um de 27×27 por bloco. **M10K** (553) são blocos de RAM de 10 Kbit. Os blocos DSP são o recurso mais escasso para multiplicações: trocá-los por lógica faz sentido quando a lógica disponível sobra, e é isso que a QP4 mede.

3 Trabalhos Relacionados

Mamba em baixa precisão. O Quamba [3, 4] quantiza Mamba-1 e Mamba-2 para 8 e 4 bits após o treino, com o *scan* em INT8 na GPU. Bi-Mamba [20], Slender-Mamba [24] e Ternary Mamba [8] usam pesos binários ou ternários nas projeções, mas mantêm o *scan* em ponto flutuante. Nenhum deles trata de como executar o decaimento sem multiplicador nem da precisão do estado inteiro.

Aceleradores de Mamba. O MARCA [14] é um acelerador ASIC simulado para Mamba-1 com multiplicadores densos e aproximações por partes das funções não lineares. O LightMamba [22] e o FastMamba [21] são aceleradores Mamba-2 para FPGAs de grande porte (Alveo, Versal, Virtex-7), com pesos de 4 e 8 bits. O LightMamba restringe a potências de dois os *fatores de re-escala* da quantização, não o coeficiente $\bar{A}_t$; o decaimento continua sendo uma multiplicação. O eMamba [13] (FPGA ZCU102 e ASIC de 22 nm) aproxima a exponencial e usa escalas PoT; o Mamba-X [23] executa o *scan* num arranjo sistólico com a exponencial numa unidade de funções especiais; o ViM-Q [18] (FPGA ZCU102) mantém o SSM em precisão mais alta. O trabalho de Fuketa e Hioki [7] é o mais próximo em objetivo: um Mamba-2 totalmente inteiro (pesos ternários, ativações INT8), treinado com QAT, em que a exponencial é trocada por potência de dois, implementado numa FPGA Ultra96-V2; pelo que descreve, os produtos elemento a elemento continuam numa unidade de multiplicação em DSP, e não há forma de complemento nem análise da precisão do estado. Fora do Mamba, o HFRWKV [17] calcula o decaimento do RWKV com uma tabela de 256 entradas. Em todos, $\bar{A}_t h$ é uma multiplicação.

Decaimento por deslocamento e precisão do estado. O trabalho mais próximo é o QMamba [15], que aproxima $\bar{A} \approx 1 - 2^{-q}$ e aplica $h - (h \gg q)$, como o ramo lento do CPoT. Ele é quantização pós-treino de modelos de visão em GPU, com aritmética simulada, estado de 4 a 8 bits com escala por grupo temporal e nenhuma análise da zona morta do deslocamento, de bits de guarda ou do arredondamento; também não há hardware. O SSDi8 [12] quantiza o SSD do Mamba-2 em 8 bits com acumulação INT32, mas o decaimento continua sendo multiplicação em ponto fixo. Trabalhos de quantização de SSMs relatam que $\bar{A}$ e o estado são as partes mais sensíveis: o PTQ4VM [5] observa que o estado em INT8 destrói a acurácia de um Vision Mamba, e o Q-S5 [1] chega à mesma conclusão com QAT em S5. Trabalhos recentes sobre o armazenamento do estado mostram que o erro de arredondamento realimentado pela recorrência é o problema: o PACE [10] guarda o estado em 4 bits com escalas alinhadas por canal, e o ReplaySSM [16] separa uma cópia de alta precisão para a recorrência. Nenhum deles trata do decaimento por deslocamento, em que o próprio deslocamento cria a zona morta, nem a corrige com bits de guarda e arredondamento.

A Tabela 1 resume a comparação. Não encontramos trabalho que meça uma unidade de *scan* sem multiplicador no decaimento contra uma com multiplicador, nem que

analise a precisão necessária do estado inteiro quando o decaimento é feito por deslocamento.

Tabela 1: Trabalhos relacionados: como cada um executa o *scan*. *Pela descrição da arquitetura no resumo.

Trabalho	**Modelo**	**Projeções**	***Scan***	**Plataforma**	**Decaimento** $\bar{A}_t h$
Quamba/Quamba2 [3, 4]	Mamba-1/2	W8A8 / W4A8	INT8	GPU	multiplicação
Bi-Mamba [20]	Mamba-1	1 bit	FP16	GPU	multiplicação
Slender-Mamba [24]	Mamba-2	1,58 bit	FP16	GPU	multiplicação
Ternary Mamba [8]	Mamba-2	1,58 bit	FP16	GPU	multiplicação
MARCA [14]	Mamba-1	INT8	FP16/INT8	ASIC (sim.)	multiplicação
LightMamba [22]	Mamba-2	W4A4	INT, escalas PoT	Alveo U280, VCK190	multiplicação
FastMamba [21]	Mamba-2	W8A8	INT, escalas PoT	Virtex-7	multiplicação
eMamba [13]	Mamba	INT	INT, exp aprox.	ZCU102, ASIC 22 nm	multiplicação
Mamba-X [23]	Vision Mamba	INT8	sistólico	ASIC (sim.)	multiplicação
ViM-Q [18]	Vision Mamba	baixa precisão	alta precisão	ZCU102	multiplicação
SSDi8 [12]	Mamba-2	W8A8	INT8, acum. INT32	GPU	multiplicação
Fuketa e Hioki [7]	Mamba-2	1,58 bit / INT8	INT, exp $\rightarrow$ PoT	Ultra96-V2	multiplicação (DSP)*
QMamba [15]	Vision Mamba	W4–8	estado 4–8 bits (simulado)	GPU	**deslocamento** (PTQ)
Este trabalho	**Mamba-2**	W8A8	**INT, estado 32b, bit-exato**	**Cyclone V (medida)**	**deslocamento** (QAT, RTL)

4 Unidade de *Scan* Proposta

A unidade proposta (`hw/rtl/scan_lane.v`, Verilog-2005) implementa a Eq. 1 para um canal x_t e seus N estados em paralelo, com a aritmética da Seção 2.4. Chamamos essa unidade de *lane*; um acelerador completo replica várias *lanes*, uma por canal processado em paralelo. A cada token chegam x_t, $\mathbf{B}_t$, $\mathbf{C}_t$ (INT8) e os códigos (m, k, j), lidos da tabela de 16 entradas da cabeça, de modo que o decaimento muda a cada passo, como exige a seletividade. A Figura 1 mostra uma das N vias de estado. São três estágios:

1. **S0**: produtos $B_t[n]\, x_t$ (INT8$\times$INT8, em DSP);
2. **S1**: recorrência. Para cada n: decaimento CPoT (deslocamento com arredondamento e subtração), injeção $((B_t x_t) \ll G) \gg j$, soma e saturação para a largura do estado. Este estágio é um laço de 1 ciclo: h_t depende de h_{t-1}, então não pode ser dividido em mais estágios sem processar canais intercalados;
3. **S2**: produtos $C_t[n]\, h_t[n]$ (em DSP), árvore de soma de N termos com um registrador por nível, deslocamento de G bits e saturação da saída para 24 bits.

Parâmetros do módulo selecionam a largura do estado, os bits de guarda, o arredondamento e o tipo de decaimento. Com `DECAY=1`, o decaimento usa um multiplicador Q1.15, $(h \cdot a + 2^{14}) \gg 15$, que é o baseline. A *lane* processa um token por ciclo; a latência é de $4 + \log_2 N$ ciclos.

5 Metodologia

Esta seção descreve cada etapa com detalhe suficiente para reprodução. Todos os scripts estão em `sw/`, e todos os números do artigo saem dos arquivos que eles geram.

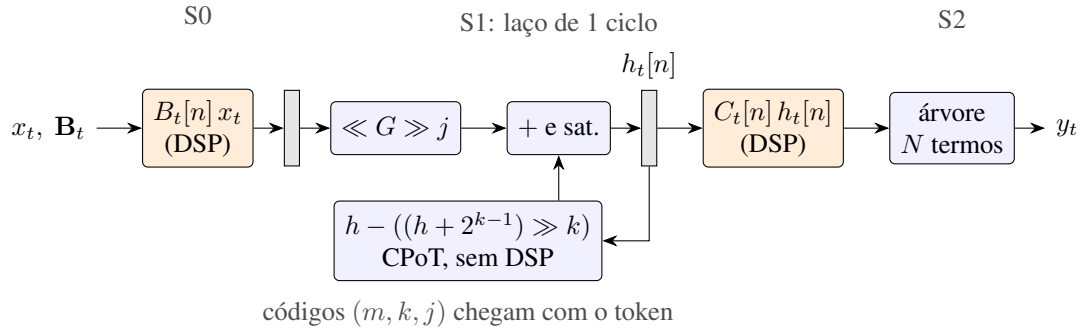


Figura 1: Uma via de estado da *lane* de *scan* (replicada $N = 16$ vezes). Laranja: multiplicações em DSP. O decaimento CPoT (azul, embaixo) usa só soma, deslocamento e subtração; no baseline, essa caixa é um multiplicador Q1.15.

5.1 Visão geral: uma especificação, três usos

Um risco comum em co-projeto é o modelo treinado e o circuito divergirem em detalhes de aritmética (larguras, arredondamentos, saturações), de modo que a acurácia medida em software não vale para o hardware. Para evitar isso, a aritmética inteira do *scan* é definida uma única vez, num *modelo de referência* em Python (`sw/models/golden_model_bitexact.py`), e esse mesmo modelo é usado de três formas (Figura 2): (a) gera as saídas esperadas para conferir o RTL; (b) substitui o *scan* em ponto flutuante na avaliação dos modelos treinados (emulação inteira); e (c) é a base dos experimentos numéricos E3 e E4. A síntese no Quartus completa o fluxo com as métricas de hardware.

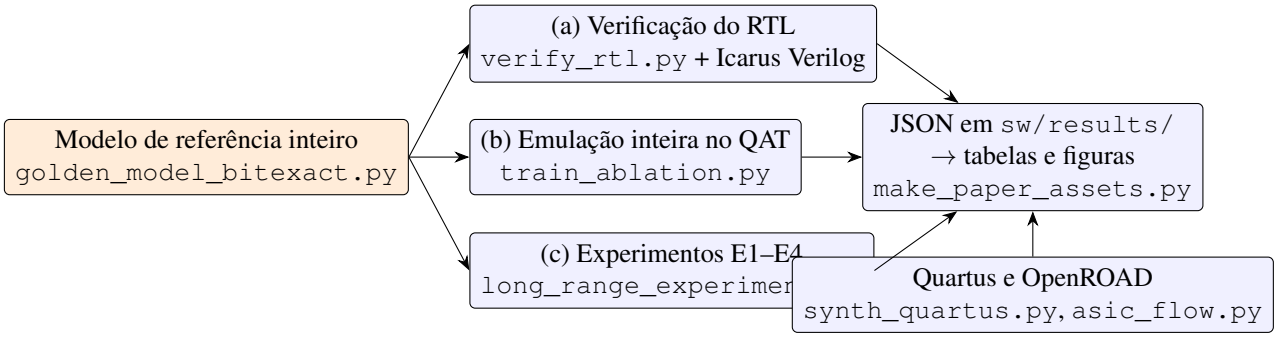


Figura 2: Fluxo metodológico. Cada etapa grava um JSON; um único script gera tabelas, figura e as macros com cada número do texto.

5.2 Modelo de referência inteiro

O modelo de referência define, bit a bit, o passo do *scan*: o produto $B_t x_t$ (INT8×INT8), a injeção com G bits de guarda e deslocamento de j bits, o decaimento (CPoT com deslocamento truncado ou arredondado, ou multiplicador Q1.15), a saturação do estado e a leitura com saturação da saída para 24 bits. Três configurações são usadas no artigo: **legado** (16 bits, $G = 0$, truncamento: a implementação ingênua), **proposta** (32 bits, $G = 10$, arredondamento) e **baseline** (32 bits, $G = 10$, multiplicador Q1.15). O próprio script tem um autoteste que reproduz o congelamento da Seção 2.4.

5.3 Verificação do RTL

O script `sw/verify_rtl.py` executa, para cada configuração da *lane*: (1) gera estímulos aleatórios com semente fixa; (2) calcula as saídas esperadas no modelo de referência; (3) grava ambos em arquivos hexadecimais; (4) compila e simula o módulo com seu *testbench* no Icarus Verilog 12, que aplica os estímulos ao circuito e compara cada saída com a esperada; e (5) registra o número de saídas comparadas e de divergências. O critério de aprovação é zero divergências. Os estímulos são 4 sequências de 512 tokens por configuração, com o estado zerado entre sequências: 85% dos tokens têm decaimento lento ($\bar{A} = 1 - 10^{-u}$, $u \sim \mathcal{U}[0{,}2;\ 3{,}5]$), 15% decaimento rápido, e j, x_t, $\mathbf{B}_t$, $\mathbf{C}_t$ são aleatórios.

Um teste que nunca falha não prova nada. Para confirmar que o teste detecta erros, simulamos um *mutante*: a *lane* proposta com o arredondamento desligado, que deve divergir do modelo de referência. A emulação inteira usada no treino é conferida contra o mesmo modelo de referência por um autoteste em `sw/models/mamba2_ssd_qat.py`.

5.4 Modelo e treinamento

Modelo. Classificador com *embedding* linear da entrada para $D = 64$ canais, 2 blocos Mamba-2 (expansão 2, isto é, 128 canais internos; $H = 4$ cabeças de $P = 32$ canais; $N = 16$; convolução de 4 posições por canal), RMSNorm final e camada linear aplicada à saída do último passo, com 68.253 parâmetros. A inicialização segue o Mamba-2: $A_h \sim \mathcal{U}[1, 16]$ e $\Delta \sim \log \mathcal{U}[10^{-3}, 10^{-1}]$. O *scan* de treino usa a forma em blocos de 64 passos do Mamba-2, matematicamente igual à recorrência sequencial (igualdade conferida por autoteste) e cerca de 3× mais rápida na GPU.

Configurações da ablação. A ablação muda só o *scan* (Tabela 2), de modo que a diferença entre duas linhas isola o efeito de uma escolha. S0 é o modelo inteiro em FP32. De S1 em diante, as projeções usam W8A8: pesos INT8 com escala por linha e ativações INT8 com escala por passo. Em S1 o *scan* recebe x, $\mathbf{B}$, $\mathbf{C}$ quantizados em INT8 com escala dinâmica e usa $\bar{A}_t$ exato: é o "*scan* ideal". A partir de S2 o *scan* obedece às restrições do hardware: as escalas de x, $\mathbf{B}$, $\mathbf{C}$ são *estáticas* e potências de dois, $2^{\lceil \log_2(m/127) \rceil}$, em que m é uma média móvel (momento 0,99) do máximo absoluto observado no treino, de modo que a requantização é um deslocamento; e Δ_t é arredondado para 2^{-j}, $j \in [0, 15]$, no domínio logarítmico. Entre S2 e S6 muda apenas a forma de $\bar{A}_t$. Todas as quantizações usam STE.

Emulação inteira. O treino usa *fake quantization*: valores arredondados, mas contas em ponto flutuante. Para medir o efeito da aritmética real do RTL, os modelos S2 e S5 já treinados são reavaliados com o *scan* substituído pela recorrência inteira do modelo de referência, sem retreinar: S2† (multiplicador Q1.15, 32 bits), S5† (CPoT, configuração proposta) e S5‡ (CPoT, estado legado de 16 bits).

Treinamento. Os hiperparâmetros estão na Tabela 3 e são iguais para todas as configurações. Não há seleção de modelo nem parada antecipada: avalia-se o modelo da última época, o que evita usar o conjunto de teste para escolher pontos de parada. A semente de cada execução fixa a inicialização e a ordem dos lotes; os conjuntos sintéticos

Tabela 2: Configurações da ablação. “Dinâm.”: escala INT8 por passo; “PoT est.”: escala estática em potência de dois.

Id	**Projeções**	$x, \mathbf{B}, \mathbf{C}$	Δ_t	$\bar{A}_t$
S0	FP32	FP32	real	exato
S1	W8A8	INT8 dinâm.	real	exato
S2	W8A8	INT8 PoT est.	2^{-j}	Q1.15 (multiplicador)
S3	W8A8	INT8 PoT est.	2^{-j}	PoT, $k \geq 1$
S4	W8A8	INT8 PoT est.	2^{-j}	PoT, $k \geq 0$
S5	W8A8	INT8 PoT est.	2^{-j}	CPoT (avaliado em RTL)
S6	W8A8	INT8 PoT est.	2^{-j}	CPoT-2 (só software)

são gerados uma única vez (semente 7) e são os mesmos para todas as execuções.

Tabela 3: Hiperparâmetros de treino (`sw/train_ablation.py`).

Otimizador	AdamW, *weight decay* 10^{-2}
Taxa de aprendizado	política *one-cycle*, máximo $2 \cdot 10^{-3}$
Recorte de gradiente	norma 1
Perda	entropia cruzada
Épocas / lote	ECG5000: 40 / 32; tarefas sintéticas: 20 / 64
Sementes	42, 100, 2026, 7, 31 (recordação com $L = 1024$: as 3 primeiras); mais 11–15 na janela e 11–17 com L
Software	Python 3.12, PyTorch 2.10, GPU NVIDIA RTX 3060 Laptop

5.5 Tarefas e métricas

ECG5000 [2]: batimentos cardíacos extraídos de um eletrocardiograma, cada um com $L = 140$ amostras, em 5 classes. Usamos a divisão oficial do arquivo UCR, com 500 séries de treino e 4.500 de teste. As classes são desbalanceadas (a majoritária tem cerca de 58% das amostras), então reportamos também a *acurácia balanceada*, a média da taxa de acerto de cada classe, que vale 20% para um classificador que sempre responde a classe majoritária.

Recordação atrasada (sintética): mede a capacidade de *reter* informação por L passos. Cada sequência tem 8 canais. Nos 8 primeiros passos, um de 4 canais recebe um pulso de amplitude 2 que indica a classe; depois, todos os canais têm apenas ruído gaussiano ($\sigma = 0{,}1$). O modelo só responde no último passo, então precisa reter a informação por $\approx L$ passos. O acaso dá 25%. Usamos $L = 256$ (4.000 sequências de treino, 1.000 de teste) e $L = 1024$ (2.000/500). Uma versão preliminar tinha também canais de ruído com $\sigma = 12$, e nela nem o modelo FP32 aprendeu a tarefa com $L \geq 128$; esses canais foram removidos.

Maioria em janela (sintética): mede a capacidade de *esquecer gradualmente*, que a recordação não testa. Cada sequência tem $L = 512$ passos e 4 canais com ruído gaussiano ($\sigma = 0{,}1$). Eventos são pulsos de amplitude 1 no canal 0 (tipo A) ou 1 (tipo B). Nos últimos $W = 64$ passos há exatamente 15 eventos em posições aleatórias, cada um A ou B com probabilidade $1/2$; o rótulo é o tipo majoritário entre eles. Antes da janela, eventos ocorrem com taxa $0{,}25$ por passo e um viés forte (85% de um tipo, sorteado por sequência, independente do rótulo). Um estado que nunca esquece ($\bar{A} = 1$) é dominado

pelos cerca de 112 eventos do histórico; um que esquece depressa ($\bar{A} \leq 0{,}5$) só vê os últimos eventos. Resolver a tarefa exige constantes de tempo da ordem da janela, $\tau \approx 16$ a 64 passos, ou seja, $\bar{A}$ entre $0{,}94$ e $0{,}985$. O acaso dá 50%; usamos 4.000 sequências de treino e 1.000 de teste.

Métricas. Reportamos acurácia de teste média $\pm$ desvio padrão entre sementes. Como a recordação tende a ser "tudo ou nada" (a semente aprende ou fica no acaso), reportamos para ela quantas sementes a resolvem, definido como acurácia acima de 90%.

Testes estatísticos e regra de parada. A hipótese principal da QP3 é que, no mesmo modelo treinado, o estado proposto (32 bits) supera o legado (16 bits) na tarefa de janela; ela é testada com teste t pareado por semente. As comparações entre configurações usam teste t de Welch, e as contagens de sementes resolvidas, teste exato de Fisher (entre configurações) ou de McNemar (pareado). Os p da família de testes da QP3 são corrigidos por Holm. Com 5 sementes a hipótese principal não atingia significância após a correção; fixamos então, antes de rodar novas sementes, uma regra de parada com até duas análises (10 e 20 sementes) e limiar de Pocock $p < 0{,}0294$ em cada uma, o que mantém o erro tipo I total em 5%. O limiar foi atingido na primeira análise, com 10 sementes.

5.6 Experimentos numéricos de memória longa

Estes experimentos não envolvem treino e isolam o efeito de cada aproximação (`sw/long_range_experiment.py`).

E1 (erro da aproximação). Para 2.000 valores de $\bar{A}$ igualmente espaçados em $[0{,}90;\ 0{,}999]$, calcula o valor quantizado $\hat{A}$ de cada esquema, o número de níveis distintos usados, o erro absoluto máximo $|\hat{A} - \bar{A}|$ e o erro relativo mediano da constante de tempo, $|\hat{\tau} - \tau|/\tau$ com $\tau = -1/\ln \bar{A}$ ($\hat{\tau} = \infty$ se $\hat{A} = 1$).

E2 (leitura após L passos). Simula 8 cabeças com τ espaçado logaritmicamente de 16 até L, 16 canais e 16 estados, um estímulo forte em $t = 0$ seguido de ruído, com L de 64 a 16.384 (5 sementes). Só $\bar{A}$ é quantizado; o resto é ponto flutuante. A métrica é a relação sinal-ruído de quantização da saída final, $\mathrm{SQNR} = 10 \log_{10}\left(\sum y^2 / \sum (\hat{y} - y)^2\right)$, em que y é a saída com $\bar{A}$ exato e $\hat{y}$ a quantizada.

E3 (impulso no estado inteiro). Injeta um impulso ($Bx = 64 \cdot 64$) no estado inteiro do modelo de referência, com $\bar{A} = 1 - 2^{-k}$ e $k \in \{4, 6, 8, 10, 12\}$, e acompanha h_t por 4τ passos. Mede a razão $h_t / h_t^{\mathrm{ideal}}$ no fim, que deve ser 1; um valor maior que 1 indica que o estado congelou.

E4 (SQNR do *datapath*). Alimenta o *scan* inteiro com entradas INT8 aleatórias por $T = 4.096$ passos ($k \in [4, 12]$, $j = k - 1$, 3 sementes) e compara com a mesma recorrência em ponto flutuante usando os *mesmos* códigos quantizados. Assim mede apenas o erro introduzido pela aritmética inteira, e não pela aproximação de $\bar{A}$.

5.7 Métricas de hardware

Cada variante da *lane* ($N = 16$) é compilada isoladamente no Quartus Prime Lite 25.1std.0 para o dispositivo `5CSEBA6U23I7` por `sw/synth_quartus.py`, com o

fluxo completo (síntese, posicionamento e roteamento, análise de *timing*). Para caracterizar o bloco sem a influência dos pinos do chip, todas as portas exceto o relógio são declaradas pinos virtuais. O relógio tem restrição de 200 MHz e o esforço de otimização é "alto desempenho". Reportamos ALMs, registradores e blocos DSP do relatório do *fitter* e a frequência máxima (F_{max}) no canto lento a 1100 mV e 100 °C, o pior caso para o grau industrial deste dispositivo. Como as variantes têm a mesma vazão (um token por ciclo), a comparação é feita em recursos e F_{max} por *lane*. A potência na FPGA é estimada pelo *Power Analyzer* do Quartus (`sw/quartus_power.py`) com o projeto recompilado a 50 MHz e taxa de chaveamento padrão de 12,5% nas entradas, propagada sem simulação (estimativa *vectorless*, que o próprio Quartus classifica como de baixa confiança).

Fluxo ASIC. O script `sw/asic_flow.py` leva cada variante pelo fluxo aberto Yosys 0.38 e OpenROAD com a biblioteca Nangate45 (FreePDK45, canto típico): síntese mapeada na biblioteca, *floorplan* com 45% de utilização, posicionamento global, `repair_design`, síntese da árvore de relógio, `repair_timing`, posicionamento detalhado e roteamento global. Timing, área e potência são extraídos com parasitas estimados do roteamento global, sem roteamento detalhado. Cada variante é executada com dois alvos de relógio: 2,5 ns, apertado demais para as variantes de 32 bits, de onde estimamos a F_{max} como $1/(T - \mathrm{WNS})$; e 4 ns (250 MHz), que todas atingem, onde comparamos área, potência e energia por passo ($P \cdot T$) na mesma frequência. A atividade das entradas é 12,5%, a mesma da FPGA.

5.8 Reprodutibilidade

A Tabela 4 lista os comandos, executados a partir de `sw/`. O script `make_paper_assets.py` gera, a partir dos JSON, as tabelas, a figura e as macros LaTeX com cada número citado no texto; nenhum número do artigo é digitado à mão.

Tabela 4: Etapas de reprodução.

Etapa	**Comando**	**Saída em `sw/results/`**
Verificação do RTL	`python3 verify_rtl.py`	`rtl_verification.json`
Experimentos E1–E4	`python3 long_range_experiment.py`	`long_range_results.json`
Ablação QAT (GPU)	`python3 train_ablation.py`	`ablation_<tarefa>.json`
Síntese Quartus	`python3 synth_quartus.py`	`quartus_dse.json`
Potência Quartus	`python3 quartus_power.py`	`quartus_power.json`
Fluxo ASIC	`python3 asic_flow.py 2.5 4.0`	`asic_nangate45.json`
Tabelas, figura e macros	`python3 make_paper_assets.py`	`paper/generated/`

6 Resultados

Os resultados estão organizados pelas questões de pesquisa, começando pelas que não dependem de treino.

6.1 QP1: aproximação do coeficiente

A Tabela 5 e a Figura 3(a) confirmam o que a Seção 2 antecipa para o PoT simples. Com $k \geq 1$, todos os valores da faixa viram $0{,}5$: o erro mediano de τ é 92,5% e o SQNR é 0 dB, ou seja, a saída não carrega nenhuma informação do estímulo. Com $k \geq 0$, todos

Tabela 5: E1 e E2: quantização de $\bar{A} \in [0{,}90;\ 0{,}999]$ (níveis distintos usados, erro absoluto máximo, erro relativo mediano de τ) e SQNR (dB) da leitura y_L com estado em ponto flutuante (média de 5 sementes). Maior SQNR é melhor.

Esquema	**Hardware**	**Níveis**	**máx** $\lvert\Delta\bar{A}\rvert$	**Erro** τ	$L = 256$	$L = 4.096$	$L = 16.384$
Q1.15 (mult.)	1 mult.	2000	0,0000	0,0%	81,3	68,8	52,9
PoT 2^{-k}, $k \geq 1$	1 shift	1	0,4990	92,5%	0,0	0,0	0,0
PoT 2^{-k}, $k \geq 0$	1 shift	1	0,1000	∞	-15,9	-17,4	-15,7
CPoT	1 shift + 1 sub.	8	0,0366	20,5%	20,7	22,3	37,5
CPoT-2 (só SW)	2 shifts + 2 sub.	92	0,0092	2,4%	50,3	51,1	41,3

viram 1 e o estado nunca esquece: a leitura dos canais lentos aponta na direção certa, mas sua magnitude cresce sem controle e o SQNR fica negativo. O CPoT usa 8 níveis na faixa e tem erro mediano de τ de 20,5%; o CPoT-2 reduz esse erro para 2,4%. O multiplicador Q1.15 continua sendo o mais preciso. **Resposta à QP1:** entre as opções só com deslocamentos, o CPoT é a única que mantém constantes de tempo longas com erro limitado.

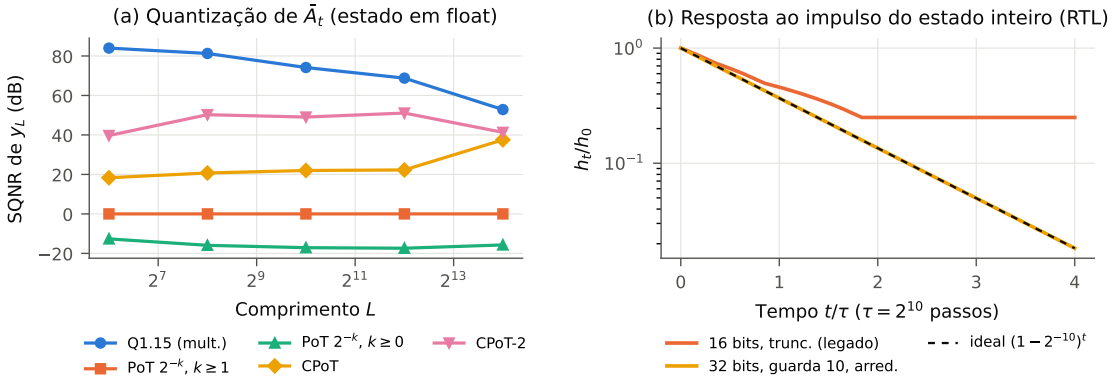


Figura 3: (a) E2: SQNR da leitura y_L em função de L (estado em ponto flutuante). (b) E3: resposta ao impulso do estado inteiro com $\bar{A} = 1 - 2^{-10}$. O estado legado de 16 bits congela pouco antes de 2τ; o estado corrigido segue a curva ideal.

6.2 QP2: precisão do estado inteiro

A Tabela 6 confirma a análise da Seção 2.4. Com o estado legado (16 bits, truncamento), depois de 4τ passos o estado vale 54,6× o ideal para $k = 12$ (e já 3,43× para $k = 8$): ele parou de decair. O SQNR do *datapath* cai para 37,0 dB. Com 32 bits, 10 bits de guarda e arredondamento, a resposta ao impulso coincide com a ideal e o SQNR sobe para 99,0 dB, no mesmo nível do baseline com multiplicador (99,3 dB). **Resposta à QP2:** o CPoT só funciona com estado de 32 bits, bits de guarda e arredondamento; a implementação ingênua de 16 bits destrói exatamente os canais de memória longa que o CPoT pretende preservar.

Tabela 6: E3 e E4: estado inteiro do RTL. Razão h_t/h_t^{ideal} após 4τ passos de um impulso com $\bar{A} = 1 - 2^{-k}$ (1,00 é o ideal) e SQNR do *datapath* inteiro com entradas aleatórias (3 sementes).

Estado	$k = 8$	$k = 10$	$k = 12$	**SQNR E4 (dB)**
Legado: 16 bits, sem guarda, truncamento	3,43	13,66	54,61	37,0
Corrigido: 32 bits, 10 bits de guarda, arredondamento	1,00	1,00	1,00	99,0
Baseline: multiplicador Q1.15, 32 bits	—	—	—	99,3

Tabela 7: Acurácia de teste (%, média $\pm$ desvio padrão entre sementes) no ECG5000 e na janela; na recordação, sementes que resolvem a tarefa (acurácia $> 90\%$) sobre sementes treinadas. S2, S4 e S5 têm 10 sementes na janela e com $L = 1024$, e S0 e S1 têm 10 com $L = 1024$; as demais células têm 5 (3 com $L = 1024$). †/‡: o mesmo modelo treinado, avaliado com o *scan* inteiro bit-exato do RTL (configuração proposta e legada).

				ECG5000 ($L = 140$)		Record. $L = 256$	Record. $L = 1024$	Janela $L = 512$
ID	Configuração	Projeções	$\bar{A}_t$	Acur.	Acur. bal.	Resolve	Resolve	Acur.
S0	FP32 (referência)	FP32	float exato	$92{,}7 \pm 0{,}3$	$52{,}0 \pm 0{,}6$	5/5	9/10	$91{,}7 \pm 1{,}0$
S1	W8A8, *scan* ideal	INT8	float exato	$92{,}7 \pm 0{,}3$	$53{,}0 \pm 1{,}0$	5/5	9/10	$91{,}8 \pm 1{,}0$
S2	*scan* HW	INT8	Q1.15 (mult.)	$92{,}8 \pm 0{,}4$	$53{,}3 \pm 1{,}1$	5/5	7/10	$90{,}4 \pm 0{,}7$
S3	*scan* HW	INT8	PoT, $k \geq 1$	$91{,}2 \pm 0{,}2$	$50{,}1 \pm 0{,}8$	0/5	0/3	$60{,}6 \pm 1{,}6$
S4	*scan* HW	INT8	PoT, $k \geq 0$	$92{,}4 \pm 0{,}3$	$52{,}1 \pm 0{,}8$	5/5	10/10	$83{,}0 \pm 2{,}9$
S5	***scan* HW**	INT8	**CPoT**	$92{,}7 \pm 0{,}4$	$52{,}9 \pm 0{,}8$	5/5	7/10	$89{,}9 \pm 1{,}0$
S6	*scan* HW	INT8	CPoT-2 (só SW)	$92{,}6 \pm 0{,}2$	$52{,}2 \pm 1{,}0$	5/5	3/3	$90{,}2 \pm 1{,}6$
S2†	S2 com *scan* inteiro bit-exato		Q1.15, 32b	$92{,}8 \pm 0{,}4$	$53{,}3 \pm 1{,}1$	5/5	7/10	$90{,}5 \pm 0{,}7$
S5†	**S5 com *scan* inteiro bit-exato**		**CPoT, 32b**	$92{,}7 \pm 0{,}4$	$52{,}9 \pm 0{,}8$	5/5	7/10	$89{,}9 \pm 0{,}9$
S5‡	S5 com estado legado (16b, trunc.)		CPoT, 16b	$92{,}6 \pm 0{,}5$	$52{,}5 \pm 0{,}3$	4/5	0/10	$83{,}4 \pm 3{,}1$

6.3 QP3: acurácia dos modelos treinados

A Tabela 7 mostra três tipos de tarefa, que respondem a perguntas diferentes.

ECG5000: dados reais, sequências curtas. Todas as configurações ficam a menos de meio ponto do FP32 (92,7%), exceto o PoT com $k \geq 1$ (91,2%), que perde todos os canais lentos. O CPoT com *scan* inteiro (S5†, 92,7%) não custa acurácia. A acurácia balanceada, perto de 52% em todas as linhas, incluindo o FP32, indica que o modelo pequeno quase não acerta as classes raras; isso é uma limitação do modelo, não do *scan*. Com $L = 140$, o ECG5000 não separa as variantes do *scan*: até o estado legado (92,6%) passa, porque os canais não chegam a ficar pequenos o bastante para congelar.

Recordação: memória longa sem esquecimento. O PoT com $k \geq 1$ não resolve nenhuma semente (acurácia de 25,1% e 24,2%; o acaso é 25%): com $\bar{A} = 0{,}5$, a pista se perde em poucos passos. Com $L = 256$, todas as outras configurações resolvem todas as sementes, *incluindo o PoT com* $k \geq 0$: um estado que nunca esquece ($\bar{A} = 1$) é ideal para guardar uma pista e ignorar o resto, e é o único que resolve as 10 sementes com $L = 1024$. Por isso a recordação sozinha não justifica o CPoT, e a tarefa de janela existe. Com $L = 1024$ o treino fica instável em todas as configurações com decaimento aprendido: resolvem 9/10 sementes o FP32, 9/10 o W8A8, 7/10 o multiplicador Q1.15 e 7/10 o CPoT. A semente 14 falha no FP32, no W8A8 e no CPoT. A diferença entre FP32 e CPoT não é significativa (Fisher, $p = 0{,}58$), e o CPoT empata com o multiplicador. As

falhas se repetem idênticas com o *scan* inteiro (S2†, S5†): são falhas de treino daquelas sementes, não da aritmética.

Maioria em janela: esquecimento gradual. É a tarefa que separa as opções de decaimento. O PoT com $k \geq 1$ esquece rápido demais (60,6%), e o PoT com $k \geq 0$ não esquece o histórico enviesado (83,0%). O CPoT atinge 89,9%: 6,9 pontos acima do PoT com $k \geq 0$ (Welch, p corrigido $= 0{,}0002$) e 0,6 ponto abaixo do multiplicador Q1.15 (90,4%, p corrigido $= 0{,}54$). Em relação ao FP32 (91,7%) a diferença é de 1,8 ponto, não significativa após a correção ($p = 0{,}016$; corrigido, $0{,}06$). O CPoT-2, com o dobro de níveis, não melhora de forma significativa (90,2%, $p = 0{,}70$): a resolução de uma oitava de τ basta para esta tarefa.

O treino representa o hardware. Em todas as tarefas e sementes, o modelo avaliado com o *scan* inteiro bit-exato (S5†, S2†) difere do avaliado com *fake quantization* em no máximo 0,1 ponto. Os números da QP3 valem, portanto, para a aritmética do RTL.

O estado legado degrada exatamente as tarefas longas. Trocar só a precisão do estado do mesmo modelo treinado (S5‡: 16 bits, truncamento) derruba a janela para 83,4%, pior que o estado de 32 bits nas 10 sementes, por 1,2 a 12,9 pontos (média 6,5; t pareado, p corrigido $= 0{,}002$, abaixo do limiar de Pocock). Na recordação com $L = 1024$, o estado legado não resolve nenhuma semente (0/10, contra 7/10 com 32 bits; McNemar exato, $p = 0{,}016$), e uma delas fica em 14%, abaixo do acaso: os canais congelados não somem, eles empurram a saída para uma resposta errada consistente. Com $L = 256$, uma das 5 sementes cai para 62%. É a confirmação, em acurácia, do que a QP2 mostra no *datapath*.

Resposta à QP3: sim, no nível do multiplicador. Com o estado de 32 bits, 10 bits de guarda e arredondamento, o *scan* CPoT mantém a acurácia FP32 no ECG5000 e na recordação com $L = 256$. Com $L = 1024$ resolve tantas sementes quanto o multiplicador (7/10), sem diferença significativa em relação ao FP32. Na janela fica 0,6 ponto abaixo do multiplicador e 1,8 abaixo do FP32, nenhuma das diferenças significativa. O CPoT supera com folga as duas variantes de potência de dois simples na tarefa que exige esquecimento gradual. Sem a precisão de estado adequada, o mesmo modelo perde 6,5 pontos na janela e deixa de resolver a recordação com $L = 1024$.

6.4 Corretude do RTL

Tabela 8: Verificação da *lane* de *scan* contra o modelo de referência inteiro (Icarus Verilog 12).

Configuração	Saídas comparadas	Divergências
CPoT, 32 bits, 10 bits de guarda, arredondamento (proposta)	2.048	0
CPoT, 16 bits, truncamento (legado)	2.048	0
Multiplicador Q1.15, 32 bits (baseline)	2.048	0
Total	**6.144**	**0**

As três configurações da Tabela 8 terminam com zero divergências, e o mutante sem arredondamento foi detectado (780 de 2.048 saídas divergentes). Junto com a emu-

lação inteira da QP3, isso liga a acurácia medida em software ao circuito: o RTL produz, bit a bit, a mesma aritmética que foi avaliada nos modelos treinados.

6.5 QP4: custo em hardware

Tabela 9: *Lane* de *scan* ($N = 16$ estados, um token por ciclo) compilada no Quartus Prime Lite 25.1std.0 para a Cyclone V `5CSEBA6U23I7`. F_{max} no canto lento (1100 mV, 100 °C). Potência dinâmica do núcleo a 50 MHz, estimativa sem simulação (12,5% de chaveamento nas entradas).

Variante	ALMs	Registradores	Blocos DSP	F_{max} (MHz)	P_{din} (mW)
CPoT, 32b + guarda (proposta)	3.299	1.432	48	68,7	62,3
CPoT, 16b, trunc. (legado, incorreto)	1.513	1.250	24	102,5	15,3
Multiplicador Q1.15, 32b (baseline)	1.873	1.712	80	64,2	29,6

A Tabela 9 mostra a troca que o CPoT faz. A *lane* proposta usa 48 blocos DSP contra 80 do baseline com multiplicador (40% a menos): os 16 multiplicadores do decaimento desaparecem, e os DSPs restantes vêm de $B_t x_t$ e $C_t h_t$, que continuam sendo multiplicações. Em troca, a lógica sobe de 1.873 para 3.299 ALMs, por causa dos 16 deslocadores variáveis de 32 bits com arredondamento. A F_{max} das duas é parecida (68,7 contra 64,2 MHz): em ambas, o caminho crítico é o laço de 1 ciclo da recorrência, que inclui o decaimento, a soma e a saturação de 32 bits. A versão legada de 16 bits é menor e mais rápida, mas, como mostra a QP2, está numericamente errada.

Na Cyclone V, com 112 blocos DSP, o número de *lanes* paralelas é limitado pelos DSPs: cabem 2 *lanes* CPoT contra 1 do baseline, e a lógica não é o limite (3.299 ALMs por *lane*, de 41.910). Para a mesma vazão, os DSPs que o CPoT libera ficam disponíveis para as projeções lineares do modelo, as maiores consumidoras de multiplicadores.

A potência na FPGA vai no sentido contrário: a potência dinâmica estimada da *lane* CPoT é 62,3 mW, contra 29,6 mW do baseline. Os deslocadores variáveis, feitos em lógica programável, gastam mais que os multiplicadores, que na FPGA são blocos dedicados e eficientes. Numa FPGA, portanto, o CPoT economiza um recurso escasso (DSP), não energia.

Tabela 10: A mesma *lane* num fluxo ASIC aberto (Yosys e OpenROAD, Nangate45, canto típico, parasitas do roteamento global). F_{max} estimada com alvo de 2,5 ns; área, potência e energia por passo (um token, 16 estados) com relógio de 250 MHz, atingido por todas as variantes.

Variante	Área (μm^2)	F_{max} (MHz)	Potência (mW)	Energia/passo (pJ)
CPoT, 32b + guarda (proposta)	72.337	306	18,5	74,0
CPoT, 16b, trunc. (legado, incorreto)	41.801	$\geq$400	11,3	45,2
Multiplicador Q1.15, 32b (baseline)	126.896	366	25,7	102,8

No ASIC (Tabela 10), onde um multiplicador não é um bloco pronto mas lógica sintetizada, o quadro se inverte. A *lane* CPoT ocupa 72.337 μm^2 contra 126.896 μm^2

(43% menos) e gasta 74,0 pJ por passo contra 102,8 pJ (28% menos). O custo é a frequência: 306 contra 366 MHz, porque o laço da recorrência passa a conter um deslocador variável de 32 bits com arredondamento, além da subtração, da soma e da saturação. A versão legada é a menor e a mais rápida, mas está numericamente errada.

Resposta à QP4: depende da plataforma. Numa FPGA, o CPoT troca 40% dos blocos DSP por mais lógica e mais potência estimada, sem perder frequência; vale a pena quando os DSPs são o limite. Num ASIC, que é o caso de uma unidade dentro de uma GPU ou NPU, o CPoT reduz área em 43% e energia por passo em 28%, com $\mathrm{F}_{\max}$ cerca de 16% menor.

7 Limitações e Próximos Passos

- **Métricas relativas.** Os números da FPGA e do ASIC servem para comparar variantes da unidade na mesma tecnologia. O fluxo ASIC usa uma biblioteca acadêmica de 45 nm, canto típico, e para no roteamento global (parasitas estimados); um processo comercial e o roteamento detalhado mudariam os valores absolutos. Não houve execução em placa.

- **Potência sem simulação.** As duas estimativas de potência usam atividade padrão propagada pelo circuito, não atividade de uma simulação com dados reais. Os deslocadores variáveis são sensíveis a essa hipótese: na FPGA, a estimativa atribui à lógica do CPoT uma taxa de chaveamento de 25%, contra 7% no baseline.

- **Frequência.** O laço de 1 ciclo limita a $\mathrm{F}_{\max}$ a cerca de 65–70 MHz nas duas variantes na FPGA; no ASIC, o laço do CPoT é cerca de 16% mais lento que o do multiplicador. Intercalar canais no laço (processar outro canal enquanto o primeiro termina) permitiria dividir a recorrência em mais estágios, e não foi explorado.

- **Escopo do modelo.** Só o *scan* foi implementado em RTL; projeções, convolução, normalizações e o controle do bloco ficam fora. Na GPU, o *scan* só é executado como recorrência na geração token a token; o treino e o processamento de entradas inteiras usam a forma em blocos, que esta unidade não acelera.

- **Escala e tarefas.** Modelos pequenos (≈68.253 parâmetros) e tarefas de classificação, duas delas sintéticas. Modelos de linguagem não foram avaliados.

- **Validade estatística.** As comparações principais usam 10 sementes, e as demais configurações, 5 (3 com $L = 1024$); isso não basta para ordenar configurações com médias próximas, como CPoT, multiplicador e FP32 na janela. A instabilidade do treino com $L = 1024$, presente também no FP32, não foi investigada (taxa de aprendizado, inicialização de A_h). Os conjuntos sintéticos usam uma semente de dados fixa; a semente só muda a inicialização e a ordem do treino.

- **Revisão da literatura.** A busca por trabalhos anteriores combinou busca web, strings booleanas no OpenAlex (que indexa IEEE, ACM e arXiv) e *snowballing* para frente; não houve acesso direto ao IEEE Xplore nem ao Scopus.

8 Conclusão

Este trabalho investigou se o multiplicador do decaimento na unidade de *scan* do Mamba-2 pode ser substituído por deslocamento e subtração. Partindo do decaimento pelo complemento em potência de dois, já usado em DSP e na quantização pós-treino do Mamba, analisamos a precisão de estado que ele exige num *scan* inteiro e avaliamos uma *lane* de *scan* em RTL, conferida bit a bit contra um modelo de referência (6.144 saídas, 0 divergências) e caracterizada numa Cyclone V. As respostas às questões de pesquisa foram:

QP1. O CPoT representa constantes de tempo longas com um deslocamento e uma subtração (erro mediano de τ de 20,5%, contra 92,5% do PoT com $k \geq 1$).

QP2. O estado precisa de 32 bits, bits de guarda e arredondamento: o estado ingênuo de 16 bits congela a memória longa e derruba o SQNR do *datapath* de 99,0 para 37,0 dB.

QP3. Com o estado proposto, o *scan* CPoT treinado com as restrições do hardware mantém a acurácia FP32 no ECG5000 e na recordação com $L = 256$, empata com o multiplicador na recordação com $L = 1024$ (7/10 sementes) e na janela (89,9% contra 90,5%) e fica 7 pontos acima da potência de dois simples; com o estado legado, o mesmo modelo cai para 83,4% na janela e não resolve nenhuma semente com $L = 1024$.

QP4. Na FPGA, a *lane* CPoT usa 40% menos blocos DSP que o baseline com multiplicador (48 contra 80), com F_{max} equivalente, mais lógica e mais potência dinâmica estimada. Em ASIC de 45 nm, a 250 MHz, ocupa 43% menos área e gasta 28% menos energia por passo, com F_{max} cerca de 16% menor.

Ficam em aberto a potência com atividade de simulação, o roteamento detalhado do fluxo ASIC, a divisão do laço da recorrência em mais estágios e a avaliação em modelos de linguagem.

Referências

[1] Steven Abreu, Jens E. Pedersen, Kade M. Heckel, and Alessandro Pierro. Q-S5: Towards quantized state space models. In *ICML Workshop on Next Generation of Sequence Modeling Architectures*, 2024.

[2] Yanping Chen, Eamonn Keogh, Bing Hu, Nurjahan Begum, Anthony Bagnall, Abdullah Mueen, and Gustavo Batista. The UCR time series classification archive. `https://www.cs.ucr.edu/~eamonn/time_series_data/` (dataset ECG5000), 2015.

[3] Hung-Yueh Chiang, Chi-Chih Chang, Natalia Frumkin, and Diana Marculescu. Quamba: A post-training quantization recipe for selective state space models. In *International Conference on Learning Representations (ICLR)*, 2025. arXiv:2410.13889.

[4] Hung-Yueh Chiang, Chi-Chih Chang, and Diana Marculescu. Quamba2: A robust and scalable post-training quantization framework for selective state space models. In *International Conference on Machine Learning (ICML)*, 2025. arXiv:2503.22879.

[5] Younghyun Cho, Changhun Lee, Seonggon Kim, and Eunhyeok Park. PTQ4VM: Post-training quantization for visual Mamba. In *IEEE/CVF Winter Conference on Applications of Computer Vision (WACV)*, 2025.

[6] Tri Dao and Albert Gu. Transformers are SSMs: Generalized models and efficient algorithms through structured state space duality. In *International Conference on Machine Learning (ICML)*, 2024. arXiv:2405.21060.

[7] Hiroshi Fuketa and Masakazu Hioki. Hardware-friendly fully quantized Mamba-2 model and its FPGA-based accelerator. *IEEE Access*, 2026.

[8] Ramprasath Ganesaraja, Sahil Dilip Panse, and Swathika N. Ternary Mamba: Grouped quantization-aware training of W1.58A16 state space models. *arXiv preprint arXiv:2606.18114*, 2026.

[9] Albert Gu and Tri Dao. Mamba: Linear-time sequence modeling with selective state spaces. In *First Conference on Language Modeling (COLM)*, 2024. arXiv:2312.00752.

[10] Patrick Haller, Filipe Laitenberger, and Alan Akbik. PACE: Periodic and channel-aligned encoding of four-bit recurrent states. *arXiv preprint arXiv:2610.29545*, 2026.

[11] Intel Corporation (Altera). *Cyclone V Device Handbook: Volume 1 – Device Interfaces and Integration*, 2023.

[12] Hyunwoo Kim, Byoungchan Ko, Minseok Kang, Minwoo Kim, Dongjin Lee, Jaehoon Lee, Sungroh Yoon, and Dahuin Jung. SSDi8: Accurate and efficient 8-bit quantization for state space duality. *arXiv preprint arXiv:2608.21952*, 2026.

[13] Jiyong Kim, Jaeho Lee, Jiahao Lin, Alish Kanani, Miao Sun, Umit Y. Ogras, and Jaehyun Park. eMamba: Efficient acceleration framework for mamba models in edge computing. *ACM Transactions on Embedded Computing Systems*, 2025.

[14] Jinhao Li, Shan Huang, Jiaming Xu, Jun Liu, Li Ding, Ningyi Xu, and Guohao Dai. MARCA: Mamba accelerator with ReConfigurable architecture. In *IEEE/ACM International Conference on Computer-Aided Design (ICCAD)*, 2024.

[15] Yinglong Li, Xiaoyu Liu, Jiacheng Li, Ruikang Xu, Yinda Chen, and Zhiwei Xiong. QMamba: Post-training quantization for vision state space models. *arXiv preprint arXiv:2501.13624*, 2025.

[16] Ze-Wei Liou, Harris Nover, Amey Naik, Andrii Skliar, Igor Shovkun, Ido Shahaf, and Tri Dao. ReplaySSM: Cache SSM inputs, not state. *arXiv preprint arXiv:2610.13778*, 2026.

[17] Shijie Liu, Zhenghao Zeng, Han Jiao, and Yihua Huang. HFRWKV: A high-performance fully on-chip hardware accelerator for RWKV. *arXiv preprint arXiv:2601.02135*, 2026.

[18] Shengzhe Lyu, Yuhan She, Patrick S. Y. Hung, Ray C. C. Cheung, and Weitao Xu. ViM-Q: Scalable algorithm-hardware co-design for vision mamba model inference on FPGA. In *IEEE Symposium on Field-Programmable Custom Computing Machines (FCCM)*, 2026. arXiv:2605.01935.

[19] Alan V. Oppenheim and Ronald W. Schafer. *Discrete-Time Signal Processing*. Pearson, 3 edition, 2010.

[20] Shengkun Tang, Liqun Ma, Haonan Li, Mingjie Sun, and Zhiqiang Shen. Bi-Mamba: Towards accurate 1-bit state space models. *Transactions on Machine Learning Research (TMLR)*, 2025. arXiv:2411.11843.

[21] Aotao Wang, Haikuo Shao, Shaobo Ma, and Zhongfeng Wang. FastMamba: A high-speed and efficient Mamba accelerator on FPGA with accurate quantization. *arXiv preprint arXiv:2505.18975*, 2025.

[22] Renjie Wei, Songqiang Xu, Linfeng Zhong, Zebin Yang, Qingyu Guo, Yuan Wang, Runsheng Wang, and Meng Li. LightMamba: Efficient Mamba acceleration on FPGA with quantization and hardware co-design. In *Design, Automation and Test in Europe (DATE)*, 2025. arXiv:2502.15260.

[23] Dongho Yoon, Gungyu Lee, Jaewon Chang, Yunjae Lee, Dongjae Lee, and Minsoo Rhu. Mamba-X: An end-to-end vision mamba accelerator for edge computing devices. *arXiv preprint arXiv:2508.02977*, 2025.

[24] Zhenxuan Yu, Takeshi Kojima, Yutaka Matsuo, and Yusuke Iwasawa. Slender-Mamba: Fully quantized Mamba in 1.58 bits from head to toe. In *International Conference on Computational Linguistics (COLING)*, 2025.